

Universidad Autónoma de la Ciudad de México

Licenciatura en Ingeniería de Software

Arquitectura de Computadoras

Práctica 07

Tiempos de Ejecución: Producto Punto en FPU, SSE y AVX

Alumno: Edson Joel Carrera Avila

Grupo: 302

Profesor: Prof. Mtro. Omar Nieto Crisóstomo

Fecha: May 24, 2026

Contents

1	Introducción	2
2	Organización del proyecto	3
3	Código fuente	5
3.1	Programa principal (main.cpp)	5
3.2	Módulo ensamblador (tiempo.asm)	6
4	Instrucciones x86/x64 utilizadas	7
4.1	FPU x87	7
4.2	SSE (128 bits, 4 floats empaquetados)	7
4.3	AVX (256 bits, 8 floats empaquetados)	8
4.4	Instrumentación de tiempo	8
5	El rol de los registros	8
5.1	Convención de llamadas Microsoft x64	8
5.2	Registros internos de las rutinas	9
6	Comparación teórica	9
7	Ejemplo de ejecución	9
7.1	Inspección de los registros AVX durante la ejecución	10
7.2	Comparación gráfica de ciclos	10
8	Repositorio GitHub	11
9	Conclusiones	11

1 Introducción

Esta práctica consiste en **medir y comparar** el tiempo de ejecución de tres implementaciones equivalentes del **producto punto** entre dos vectores de números **float**: una usando la **FPU x87** (un dato a la vez), otra con **SSE** (cuatro datos por instrucción, 128 bits) y la tercera con **AVX** (ocho datos por instrucción, 256 bits). Las tres rutinas están escritas en **ensamblador x64 nativo** de Windows (MASM) y se invocan desde un programa principal en **C++**.

El objetivo no es únicamente reproducir el cálculo, sino **cuantificar** la ganancia de rendimiento que ofrecen las extensiones SIMD (*Single Instruction, Multiple Data*) respecto al modelo escalar tradicional de la FPU. Para ello se utiliza la instrucción **RDTSC** (*Read Time-Stamp Counter*), que devuelve el número de ciclos transcurridos en el procesador, complementada con barreras de serialización **LFENCE** que impiden el reordenamiento especulativo.

¿Qué es el producto punto?

Dados dos vectores $\vec{a} = (a_1, a_2, \dots, a_N)$ y $\vec{b} = (b_1, b_2, \dots, b_N)$ de dimensión N , su **producto punto** (o producto escalar) se define como:

$$\vec{a} \cdot \vec{b} = \sum_{i=1}^N a_i b_i = a_1 b_1 + a_2 b_2 + \dots + a_N b_N \quad (1)$$

En esta práctica usamos $N = 1000$ con $a_i = 1.5$ y $b_i = 2.0$, lo que da:

$$\vec{a} \cdot \vec{b} = 1000 \cdot (1.5 \cdot 2.0) = 3000.0 \quad (2)$$

¿Qué son FPU, SSE y AVX?

Tecnología	Ancho	Floats por instrucción
FPU x87	80 bits (un dato)	1
SSE	128 bits (XMM)	4
AVX	256 bits (YMM)	8

Table 1: Capacidad SIMD de las tres tecnologías comparadas

Teóricamente, SSE debe ser hasta $4\times$ más rápido que la FPU, y AVX hasta $8\times$. En la práctica intervienen otros factores: latencia del acceso a memoria, predicción de saltos y eficiencia del reordenamiento del procesador.

¿Cómo medimos el tiempo?

La instrucción **RDTSC** expone el contador de ciclos del procesador. Su uso correcto requiere encerrar la llamada con instrucciones **LFENCE** para evitar que el procesador adelante o

retrase su lectura por ejecución especulativa:

$$\text{LFENCE} \rightarrow \text{RDTSC}_{\text{inicio}} \rightarrow \text{LFENCE} \rightarrow \text{rutina} \rightarrow \text{LFENCE} \rightarrow \text{RDTSC}_{\text{fin}}$$

La diferencia $\text{RDTSC}_{\text{fin}} - \text{RDTSC}_{\text{inicio}}$ es el número de ciclos consumidos por la rutina.

2 Organización del proyecto

El proyecto se compila como una aplicación de consola **x64** en Visual Studio 2022 con el conjunto de herramientas v143. La plataforma debe ser obligatoriamente **x64** (no **Win32**) porque las rutinas usan la convención de llamadas de 64 bits y las extensiones AVX.

Archivo	Función
<code>tiempo.asm</code>	Implementa en ensamblador x64 las tres versiones del producto punto: <code>productoPunto_fpu</code> , <code>productoPunto_sse</code> y <code>productoPunto_avx</code> . Cada rutina recibe punteros a los vectores y la dimensión N , y devuelve el resultado en <code>XMM0</code> .
<code>main.cpp</code>	Programa principal en C++. Inicializa dos vectores de 1000 <code>float</code> alineados a 32 bytes, invoca cada rutina y mide su costo en ciclos con <code>RDTSC + LFENCE</code> .
<code>Practica07_TiemposEjecucion.vcxproj</code>	Proyecto de Visual Studio con MASM habilitado (<code>masm.props</code> / <code>masm.targets</code>) y conjunto de instrucciones avanzado (<code>/arch:AVX</code>) para que el compilador C++ acepte los intrínsecos.

Table 2: Archivos del proyecto

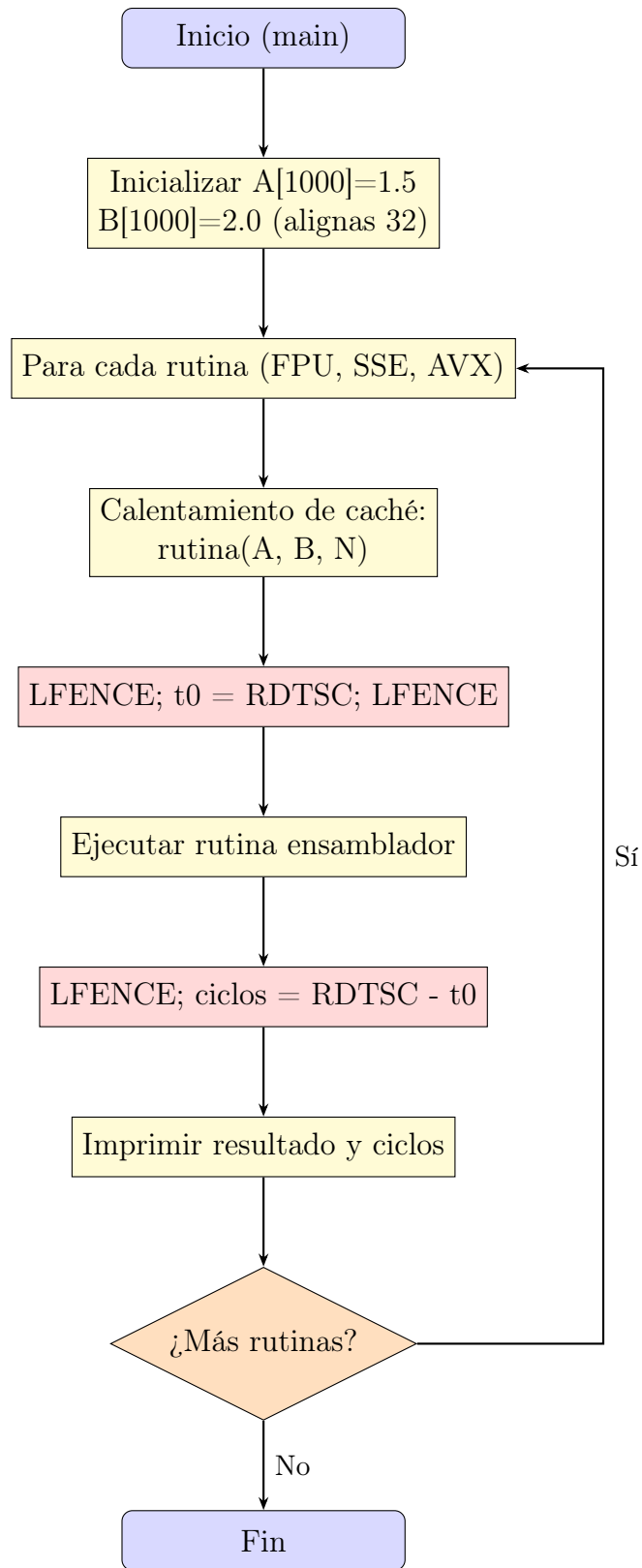


Figure 1: Diagrama de flujo de la medición de tiempos

3 Código fuente

3.1 Programa principal (main.cpp)

El programa principal reserva los vectores con `alignas(32)` (requisito de AVX), los inicializa, y mide el costo de cada rutina con `RDTSC+LFENCE`. Para evitar repetir el bloque de medición, las tres rutinas se almacenan en un arreglo de punteros a función.

```

1 #include <stdio>
2 #include <intrin.h>
3
4 extern "C" float productoPunto_fpu(const float*, const float*, size_t);
5 extern "C" float productoPunto_sse(const float*, const float*, size_t);
6 extern "C" float productoPunto_avx(const float*, const float*, size_t);
7
8 int main() {
9     alignas(32) static float A[1000];
10    alignas(32) static float B[1000];
11
12    for (int i = 0; i < 1000; i++) {
13        A[i] = 1.5f;
14        B[i] = 2.0f;
15    }
16
17    using Func = float (*)(const float*, const float*, size_t);
18    Func rutinas[] = { productoPunto_fpu, productoPunto_sse,
19                      productoPunto_avx };
20    const char* nombres[] = { "FPU", "SSE", "AVX" };
21
22    printf("Calculando Producto Punto (N = 1000)\n");
23    printf("-----\n");
24
25    for (int i = 0; i < 3; i++) {
26        rutinas[i](A, B, 1000); // calentamiento
27
28        _mm_lfence();
29        unsigned __int64 inicio = __rdtsc();
30        _mm_lfence();
31
32        float res = rutinas[i](A, B, 1000);
33
34        _mm_lfence();
35        unsigned __int64 ciclos = __rdtsc() - inicio;
36
37        printf("%s | Res: %.1f | Tiempo: %llu ciclos\n", nombres[i], res,
38              , ciclos);
39    }
40    return 0;
41 }

```

Listing 1: main.cpp — medición con RDTSC y LFENCE

3.2 Módulo ensamblador (tiempo.asm)

El archivo `tiempo.asm` contiene las tres rutinas declaradas con `PUBLIC` para que el linker pueda resolverlas desde C++. Todas siguen la convención de llamadas Microsoft x64: `RCX`=primer puntero, `RDX`=segundo puntero, `R8`=dimensión, retorno en `XMM0`.

```

1 productoPunto_fpu PROC
2     xor    r9, r9                ; i = 0
3     fldz                     ; ST(0) = 0.0 (acumulador)
4 L1_fpu:
5     cmp    r9, r8
6     jge    L2_fpu
7     fld    DWORD PTR [rcx + r9*4] ; ST(0) = vecA[i]
8     fmul   DWORD PTR [rdx + r9*4] ; ST(0) *= vecB[i]
9     faddp  st(1), st(0)          ; acum += ST(0); pop
10    inc    r9
11    jmp     L1_fpu
12 L2_fpu:
13    sub     rsp, 8
14    fstp    DWORD PTR [rsp]      ; guarda ST(0) en pila
15    movss   xmm0, DWORD PTR [rsp] ; copia a XMM0 (retorno)
16    add     rsp, 8
17    ret
18 productoPunto_fpu ENDP

```

Listing 2: `productoPunto_fpu` — versión FPU x87 (1 float por iteración)

```

1 productoPunto_sse PROC
2     xorps   xmm0, xmm0          ; acumulador = (0, 0, 0, 0)
3     xor     r9, r9
4 L1_sse:
5     cmp     r9, r8
6     jge     L2_sse
7     movaps  xmm1, XMMWORD PTR [rcx + r9*4] ; carga 4 floats de vecA
8     mulps   xmm1, XMMWORD PTR [rdx + r9*4] ; *= 4 floats de vecB
9     addps   xmm0, xmm1          ; acumula 4 productos
10    add     r9, 4
11    jmp     L1_sse
12 L2_sse:
13    haddps   xmm0, xmm0          ; reducción horizontal
14    haddps   xmm0, xmm0          ; resultado escalar en XMM0
15    ret
16 productoPunto_sse ENDP

```

Listing 3: `productoPunto_sse` — versión SSE de 128 bits (4 floats por iteración)

```

1 productoPunto_avx PROC
2     vxorps  ymm0, ymm0, ymm0    ; acumulador YMM = 8 ceros
3     xor     r9, r9
4 L1_avx:
5     cmp     r9, r8
6     jge     L2_avx
7     vmovaps ymm1, YMMWORD PTR [rcx + r9*4] ; carga 8 floats de vecA
8     vmulps  ymm1, ymm1, YMMWORD PTR [rdx + r9*4] ; *= 8 floats de vecB

```

```

9      vaddps ymm0, ymm0, ymm1                ; acumula 8 productos
10     add r9, 8
11     jmp L1_avx
12 L2_avx:
13     vextractf128 xmm1, ymm0, 1              ; reduce 256 -> 128 bits
14     vaddps xmm0, xmm0, xmm1
15     vhaddps xmm0, xmm0, xmm0                ; reducción horizontal
16     vhaddps xmm0, xmm0, xmm0
17     vzeroupper                             ; limpia estado AVX
18     ret
19 productoPunto_avx ENDP

```

Listing 4: productoPunto_avx — versión AVX de 256 bits (8 floats por iteración)

4 Instrucciones x86/x64 utilizadas

4.1 FPU x87

Instrucción	Operación
FLDZ	Carga 0.0 al tope de la pila FPU.
FLD	Carga un float (4 bytes) desde memoria a ST(0).
FMUL	Multiplica ST(0) por un operando de memoria.
FADDP	Suma ST(0) a ST(1) y hace <i>pop</i> .
FSTP	Guarda ST(0) en memoria y hace <i>pop</i> .
MOVSS	Copia un float (<i>scalar single</i>) entre memoria y XMM.

Table 3: Instrucciones x87 usadas en productoPunto_fpu

4.2 SSE (128 bits, 4 floats empaquetados)

Instrucción	Operación
XORPS	XOR empaquetado: limpia los 128 bits de un registro XMM.
MOVAPS	<i>Move Aligned Packed Single</i> : carga 4 floats alineados desde memoria.
MULPS	Multiplica 4 floats por 4 floats (elemento a elemento).
ADDPS	Suma 4 floats con 4 floats.
HADDPS	Suma horizontal: suma pares adyacentes dentro del registro XMM.

Table 4: Instrucciones SSE usadas en productoPunto_sse

4.3 AVX (256 bits, 8 floats empaquetados)

Instrucción	Operación
VXORPS	Versión AVX de XOR (limpia un registro YMM completo).
VMOVAPS	Carga 8 floats alineados a 32 bytes desde memoria.
VMULPS	multiplica 8 floats por 8 floats.
VADDPS	Suma 8 floats con 8 floats.
VEXTRACTF128	Extrae la mitad alta de un YMM a un XMM (reducción 256→128).
VHADDPS	Suma horizontal AVX.
VZEROUPPER	Limpia la mitad alta de los registros YMM (evita penalización AVX→SSE).

Table 5: Instrucciones AVX usadas en `productoPunto_avx`

4.4 Instrumentación de tiempo

Instrucción	Operación
RDTSC	<i>Read Time-Stamp Counter</i> : devuelve en <code>EDX:EAX</code> el contador de ciclos del procesador desde su último <i>reset</i> .
LFENCE	<i>Load Fence</i> : serializa la ejecución; impide que las cargas posteriores se ejecuten antes de que las anteriores terminen.

Table 6: Instrucciones de medición usadas en `main.cpp` a través de intrínsecos

5 El rol de los registros

5.1 Convención de llamadas Microsoft x64

A diferencia de la *cdecl* de 32 bits (parámetros en la pila), en x64 los primeros cuatro argumentos enteros viajan en registros. Esta práctica solo usa los tres primeros:

Registro	Parámetro	Descripción
RCX	<code>const float* vecA</code>	Puntero al primer vector.
RDX	<code>const float* vecB</code>	Puntero al segundo vector.
R8	<code>size_t N</code>	Dimensión de los vectores (1000).
XMM0	retorno	La parte baja contiene el <code>float</code> resultado.

Table 7: Mapa de parámetros para la convención Microsoft x64

5.2 Registros internos de las rutinas

Registro	Rutina	Rol
R9	las tres	Índice de iteración i .
ST(0), ST(1)	FPU	Acumulador y operando temporal en la pila FPU.
XMM0	SSE	Acumulador empaquetado de 4 floats.
XMM1	SSE	Carga temporal de 4 floats desde memoria.
YMM0	AVX	Acumulador empaquetado de 8 floats.
YMM1	AVX	Carga temporal de 8 floats.

Table 8: Registros internos usados por cada rutina

6 Comparación teórica

Cada rutina realiza la misma cantidad de **trabajo lógico** (1000 multiplicaciones y 1000 sumas), pero el **número de iteraciones** del bucle difiere:

Rutina	Floats/iter.	Iteraciones (N=1000)	Speedup teórico
FPU x87	1	1000	$1 \times$ (referencia)
SSE	4	250	$4 \times$
AVX	8	125	$8 \times$

Table 9: Iteraciones y *speedup* teórico para $N = 1000$

En la práctica, el *speedup* medido es **menor** que el teórico por:

- Latencia de las propias instrucciones SIMD (algunas son más caras que su contraparte escalar).
- Costo fijo de la **reducción horizontal** (HADDPS, VEXTRACTF128).
- Acceso a memoria limitado por el ancho de banda L1.
- Sobrecarga del prólogo, epílogo y `vzeroupper`.

7 Ejemplo de ejecución

Tras compilar en **Release** | **x64** y ejecutar el programa, la consola muestra el resultado de las tres rutinas y los ciclos consumidos por cada una. El resultado debe ser `3000.0` en las tres (correctitud), y los ciclos deben respetar la jerarquía **FPU > SSE > AVX**:

```

Consola de depuración de Mi
Calculando Producto Punto (N = 1000)
-----
FPU | Res: 3000.0 | Tiempo: 6732 ciclos
SSE | Res: 3000.0 | Tiempo: 900 ciclos
AVX | Res: 3000.0 | Tiempo: 612 ciclos
-----
C:\Users\ING_EJCA\Desktop\Practica07_TiemposEjecucion\proyecto\x64\Debug\Practica07_TiemposEjecucion.exe (proceso 5212) se cerró con el código 0 (0x0).
Presione cualquier tecla para cerrar esta ventana. . .
  
```

Figure 2: Salida en consola del programa para $N = 1000$ con $A_i = 1.5$ y $B_i = 2.0$. Las tres rutinas devuelven 3000.0; los ciclos descenden de FPU a SSE a AVX, mostrando la ganancia del paralelismo SIMD.

7.1 Inspección de los registros AVX durante la ejecución

Para verificar que los registros YMM contienen los 8 floats esperados, se coloca un *breakpoint* dentro del bucle de `productoPunto_avx` y se observa el contenido de YMM0 (acumulador) y YMM1 (carga temporal) en la ventana *Registers* de Visual Studio:

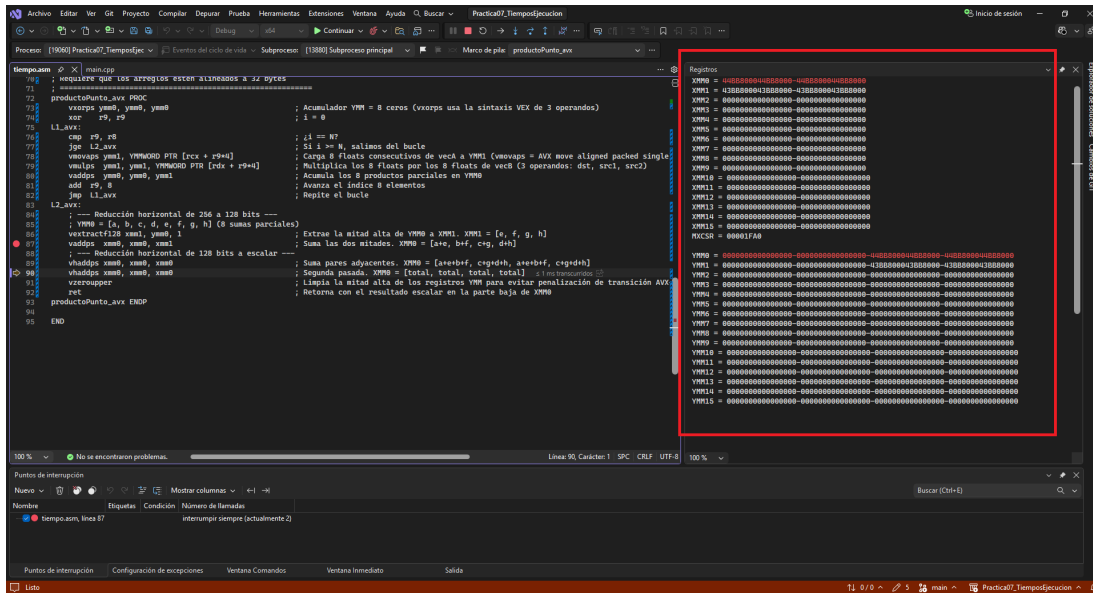


Figure 3: Ventana *Registers* de Visual Studio durante el bucle AVX: YMM1 contiene 8 productos $1.5 \times 2.0 = 3.0$ y YMM0 acumula los productos parciales en sus 8 carriles.

7.2 Comparación gráfica de ciclos

Al ejecutar el programa varias veces y promediar los ciclos, se observa con claridad que AVX es el más rápido, seguido por SSE y, en último lugar, la FPU:

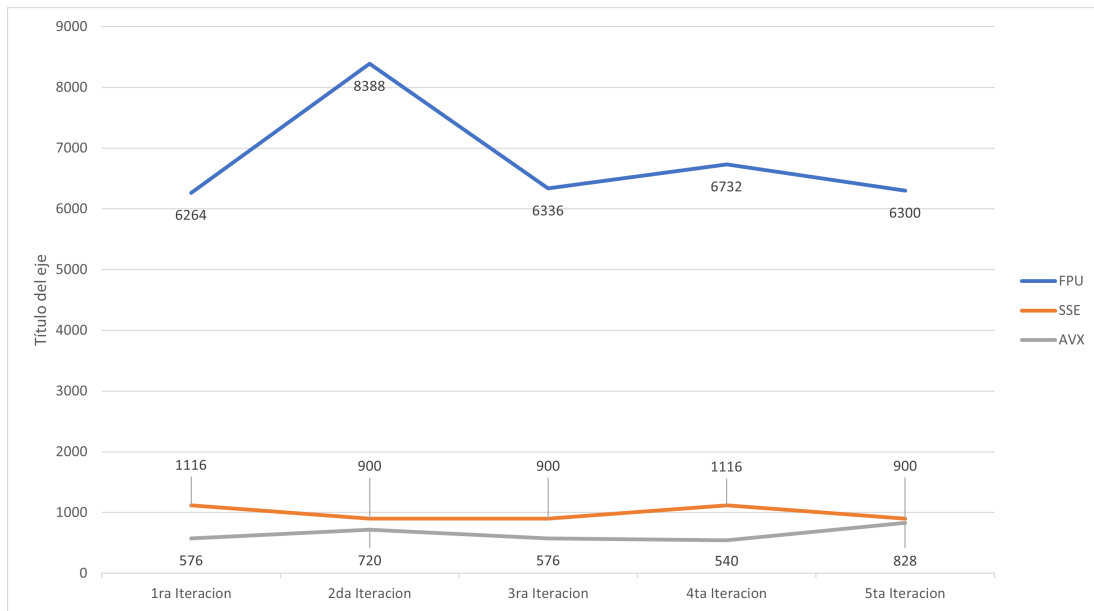


Figure 4: Comparación de ciclos promedio para las tres rutinas (5 ejecuciones, $N = 1000$). El paralelismo SIMD reduce sustancialmente el costo del producto punto.

8 Repositorio GitHub

https://github.com/7mo-ArquitecturaComputadoras/Practica07_TiemposEjecucion

9 Conclusiones

- La comparación FPU vs SSE vs AVX demuestra en términos cuantitativos el valor del **paralelismo a nivel de datos**: una sola instrucción **VMULPS** sustituye a ocho **FMUL** consecutivos, lo que reduce drásticamente el número de instrucciones decodificadas, despachadas y retiradas por el procesador.
- El uso de **RDTSC** entre **LFENCE** es esencial para medir con precisión: sin las barreras, el procesador puede mover instrucciones a través del **RDTSC** y producir lecturas inconsistentes. El calentamiento previo de la caché L1 también es necesario, pues la primera invocación incluye el costo de los fallos de caché.
- La **reducción horizontal** (**HADDPS**, **VEXTRACTF128** + **VHADDPS**) es un sobre costo inherente a SIMD: el acumulador termina con varios valores parciales que es necesario sumar entre sí. Para vectores grandes este costo se amortiza, pero para vectores pequeños puede dominar.
- La instrucción **VZEROUPPER** al final de la rutina AVX evita la **penalización de transición** entre los modos AVX y SSE: si se omitiera, una llamada posterior a

código SSE puro sufriría una latencia adicional de varios cientos de ciclos en algunas microarquitecturas.

- La alineación de memoria a 32 bytes (`alignas(32)`) es **obligatoria** para las cargas alineadas `VMOVAPS`; usar `VMOVUPS` (sin alineación) habría permitido tolerar arreglos mal alineados, pero a costa de microoperaciones extra en algunos procesadores. La elección refleja un principio común en programación SIMD: si tienes control sobre los datos, alíñalos.